19. Управление на качеството на софтуерни приложения. Тестване на софтуер.
Изложението по въпроса трябва да включва следните по-съществени елементи:
1. Осигуряване на качеството – качество на софтуера, алтернативи за осигуряване на качество.
2. Тестови дейности, управление и автоматизация
3. Видове тестване - тестване с контролни списъци, тестване с класове на еквивалентност, разделяне на входния домейн и тестване на границите.
4. Нива на тестване и приложение на техниките за тестване.
5. Измерване. Метрики – характеристики и класификация. Метрики за качество на софтуерен продукт и софтуерен процес

1. **Осигуряване на качеството – качество на софтуера, алтернативи за осигуряване на качество**.

Осигуряването на качество изисква да се изпълнят набор от дейности (например тестване), проектирани да оценят процеса, спрямо който се създават и поддържат продуктите. Целта е да се докаже качеството, т.е. правилното поведение, както и да се установят и отстранят проблеми.

Качество на софтуера се дефинира като степента, до която система, компонент или софтуерен процес отговаря на специфицираните изисквания и удовлетворява нуждите или очакванията на заинтересованите лица.

Алтернативи за осигуряване на качеството са софтуерна разработка (ориентирана към процеса) и софтуерна поддръжка (ориентирана към продукта).

2. Тестови дейности, управление и автоматизация

Основните тестови дейности са:

- **Планиране на тестовете** – дефиниране на тестова цел, планиране на ресурси, персонал и избор на формални модели и техники за тестване.
- **Конструиране на тестов модел** – идентифициране на източник на информация и събиране на данни, както и анализ и създаване на модела.
- **Генериране на тестови сценарии от модела**, където:
    - **Тестови сценарий (Test case)** дефинираме като колекция от елементи и свързана с тях информация, осигуряващи изпълнението на тест или тестова серия.
    - **Тестова серия (Test run)** е динамична единица от специфични тестови дейности в общата тестова последователност върху избран тестов обект. На прост език изпълнение на няколко теста.
- **Създаване и управление на тестови пакети (test suites)**, където:
    - Тестов пакет (Test suite) наричаме колекция от отделни тестови сценарии, които се стартират в тестова последователност докато не се удовлетвори даден критерий за спиране.
- **Подготовка на тестова процедура**, която определя последователността и превключването на тестовите серии.
- **Изпълнение на тестовете** и наблюдение, което се състои от:
    - Заделяне на време и ресурси;
    - Стартиране и изпълнение на тестове и събиране на резултати от изпълнението им;

    - Проверка на тестовите резултати чрез тестови оракули.
- **Анализ и проследяване** – анализиране на индивидуални тестови сценарии и проверка на резултатите от тях с цел идентифициране на повреди.

Управление:
Съществуват следните видове организация и управление на екипа за тестване:

- **Вертикален модел на организация** – организацията е около продукта, като една или повече тестови задачи се асоциират с определен изпълнител.
- **Хоризонтален модел на организация** – за големи организации, като един тестов екип изпълнява един тип тестване за всички продукти.
- **Смесен модел на организация**, където:
    - ниските нива на тестване се изпълняват от екипите отговарящи за съответните проекти
    - системното тестване се споделя между подобни проекти;
    - съществува обща поддръжка на проекта от осигурена централна единица.

Автоматизация:
Целта на автоматизацията е да освободи тестерите от досадни и повтарящи се задачи и да повиши тестовата производителност. Важни въпроси касаещи автоматизацията са възможността за автоматизация на специфични тестови сценарии, изборът на достъпни софтуерни инструменти за автоматизация и крайната цена на автоматизацията.

Тестова дейност	Възможност за автоматизация
Изпълнение	
Изпълнение на тестове	Висока
Планиране и подготовка	
Генериране на тестови сценарии	Висока
Създаване на формални модели	Средна
Подготовка на тестови сценарии	Средна
Цялостно планиране на тестовия процес	Ниска
Планиране на тестовата процедура	Ниска
Анализ и проследяване	
Анализ на надеждността	Висока
Анализ на тестовото покритие	Висока
Действия за подобряване на продукта	Ниска

3. Видове тестване
    a. **Тестване с контролни списъци**

Ad-hoc тестването представлява стартиране на софтуера, наблюдение на поведението и идентифициране на проблемите. Тестването с **контролни списъци** представлява създаване на неформални **TODO** списъци за проследяване на изтестваните по ad-hoc начин елементи.

Съществуват следните типове контролни списъци:
- **Базови** – прости списъци от елементи, които трябва да се изтестват;
- **Йерархични** – списъците са разделени на нива, като елементите от по-горните нива съдържат списъци от по-ниски нива;
- **Комбинирани** (многомерни) – списъците са многомерни, като всеки контролен списък се обхожда за всички елементи от останалите контролни списъци;
- **Смесени** – съчетание на йерархичните и комбинираните контролни списъци.

При контролните списъци се срещат следните проблеми и ограничения:
- **Трудности при покриване на всички функционалности** от различни гледни точки и нива на грануларност;
- **Припокриване на елементи** в различни контролни списъци;
- **Трудност при описване на сложни взаимодействия** между различни компоненти на системата

.

    b. **Тестване с класове на еквивалентност**

От дискретната математика знаем, че класовете на еквивалентност породени от дадена релация $\equiv_A$ на еквивалентност над множество A, представляват разбиване на това множество. С други думи, ако $C_1, \ldots, C_n$ е разбиване на множеството A, за някое $n \in \mathbb{N}$, то:

- $\forall i, j, i \neq j$ е изпълнено, че $C_i \cap C_j = \emptyset$;
- $\bigcup_{i=1}^{n} C_i = A$;
- $C_i = [x]_A = \{a \in A \mid x \equiv_A a\}$.

Практическата импликация при тестването е, че тестовите сценарии (или входа им) се разбиват на класове на еквивалентност спрямо очаквания тип на взаимодействие. Може да се раздели на следните стъпки:
- Дефиниране на класове на еквивалентност
- Избор на един тестов сценарии за всеки клас на еквивалентност
- Постигане на пълно покритие на класовете на еквивалентност

Типовете разделяне на класове на еквивалентност са:
- Базирано на софтуерни елементи;
- Базирано на определени свойства, релации, логически условия;
- Базирано на комбинация от горните две

    c. **Разделяне на входния домейн и тестване на границите**

Генериране на тестови сценарии посредством присвояване на специфични стойности на входните променливи въз основа на анализ на входния домейн

Тестване с разделяне на входния домейн наричаме покриване на малък брой входни ситуации посредством систематичен избор на определени входни стойности.
Характеризира се с:

- Тестване на входно/изходните зависимости – осигуряване на стойности за всички входни променливи;
- Изходните променливи не се специфицират експлицитно – проверяваме дали изход се получава от вход;
- Прилага се главно за функционално тестване;
- Детайлите по реализацията могат да се използват за анализ на входните променливи, което е предпоставка за извършване на структурно тестване.

Разделя се на следните стъпки:

- Идентифициране на входното пространство и дефиниране на входен домейн.
- Разделяне на входния домейн на поддомейни;
- Анализ на поддомейните с цел определяне на границите им по всички измерения;
- Избор на всички тестови точки (сценарии) покриващи поддомейните;
- Тестване с избраните тестови точки, решаване на проблеми и анализиране на разултатите

Проблемите при разделянето на входния домейн са:

- неопределеност на даден вход – тестваната програма не обработва някои входни стойности;
- противоречивост за даден вход – повреда в системата или производство на различни изходи при един и същ вход;

Проблеми при определянето на границите на поддомейните са:

- Проблем със затвореността на границите – отворена граница се специфицира като затворена;
- Изместване на границата;
- Липсваща граница;
- Излишна граница.

d. други
    i. тестване с покритие на класове
    ii. тестване с машина на крайните състояния
    iii. тестване с граф на данновия поток

4. Нива на тестване и приложение на техниките за тестване.

Откъм йерархична гледна точка тестовете са:

- **Тестване на ниво програмна единица (Unit testing)** – тестване на малки програмни единици, като функция, метод или най-много един клас, по метода на бялата кутия. Представлява най-ниското ниво на абстракция в йерархията на тестовете. Изпълняват се от разработчиците.
- **Интеграционно тестване (Integration testing)** – Тестване на взаимодействието между различни модули и компоненти, както по метода на бялата, така и на черната кутия.

- **Системно тестване (System testing)** –Тестване на цялата система като едно цяло. по метода на черната кутия.
- **Тестване за приемане на системата (Acceptance testng)** – тестване готовността на продукта за доставяне на крайните потребители. Извършва се срещу среда където са разгърнати всички компоненти на системата като настъпва в края на системното тестване. При процеси като scrum/kanban се извършва поне веднъж в края на всеки sprint, като част от тестовия цикъл, преди да се направи release.

Някои други видове тестове са:
- Бета тестване
- Regression testing
- Performance testing
- Load testing

5. Измерване. Метрики – характеристики и класификация. Метрики за качество на софтуерен продукт и софтуерен процес

Измерване наричаме процес, при който в съответствие с определени правила, на характеристиките на изследвания обект се съпоставят стойности.

Дейности при измерването са:
- Формулиране на метрична система;
- Събиране на данни;
- Анализиране;
- Интерпретиране;
- Връщане на обратна връзка.

Мярка наричаме стойност съпоставена на някое измерване. Следователно тя показва състоянието на измерваната характеристика

Метрика на дадена характеристика, наричаме функцията съпоставяща нейните състояния на мерките им.

Характеристики на метриките:
- Надеждна (Еднозначна) – еднакви резултати при еднакви условия;
- Уместна – измерва значим (нетривиален) атрибут;
- Валидна – измерва искания атрибут
- Неподатлива на предубедени намеси от заинтересовани страни;
- Взаимно изключваща – не измерва вече измерен атрибут
- Сравнимост: Резултатите от измерванията трябва да могат да се сравняват с други резултати или стандарти.
- Практичност: Метриките трябва да бъдат лесни за събиране и анализ.

Класификация на метриките:
- **Процесни метрики**: Измерват ефективността и качеството на процесите в софтуерното разработване
- **Продуктови метрики**: Оценяват качествата на софтуерния продукт.

- **Проектни метрики**: Свързани с управлението на проекта, включително време, разходи и ресурси.

Метрики за качество на софтуерен процес:

- **Брой открити дефекти на етап**: Брой дефекти, открити в различни етапи на разработка.
- **Средно време за изпълнение на задачи**: Средното време, необходимо за завършване на отделни задачи.
- **Процент на завършени задачи в срок**: Процент на задачите, завършени в определените срокове.
- **Брой завършени задачи**: Брой задачи, успешно завършени за определен период.

Метрики за качеството на софтуерния продукт

- **Средно време до отказ (MTTF - Mean time to failure);**
- **Средно време за възстановяване (MTTR - Mean Time To Repair)**: Средното време, необходимо за възстановяване на системата след отказ.
- **Процент грешки (Defect rate);**
- Брой проблеми идентифицирани от потребителите (**Customer problems**);
- Удовлетвореност на потребителите (**Customer satisfaction**)
- **Време за обучение**: Времето, необходимо на нов потребител да овладее използването на системата.
- **Използване на ресурси**: Количеството ресурси (CPU, памет), използвани от системата.